

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
дисциплины «Программирование на Python»

Вариант 20

Выполнил:
Скоробогатов Виктор Андреевич
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Тема: Работа множествами и словарями в языке Python

Цель: Приобретение навыков по работе с множествами и словарями при написании программ с помощью языка программирования Python версии 3.x.

Ход работы

Ссылка на репозиторий:

https://github.com/LostsSs78/NCFU-Python_25-12-24-15-44

Для выполнения работы был создан новый общедоступный репозиторий на GitHub, использующий лицензию MIT и язык программирования Python, далее он был клонирован на персональный компьютер:

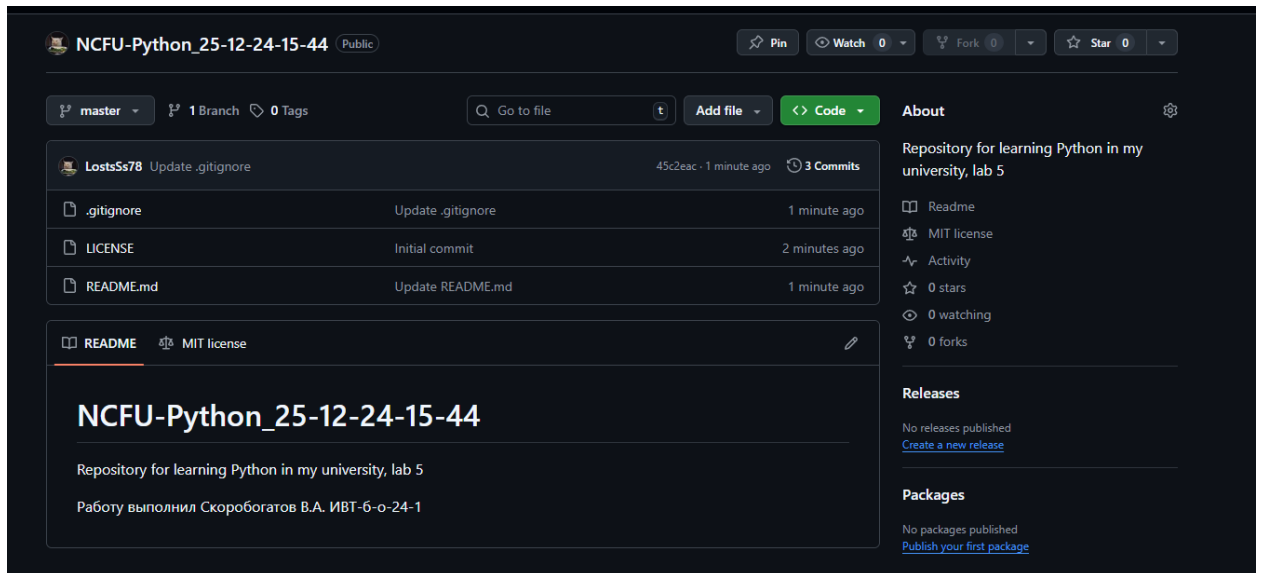


Рисунок 1. Репозиторий на GitHub

Далее были и проработаны примеры, созданы отдельные модули для каждого примера и зафиксированы изменения в репозитории:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

if __name__ == "__main__":
    # Определим универсальное множество
    u = set("abcdefghijklmnopqrstuvwxyz")

    a = {"b", "c", "h", "o"}
    b = {"d", "f", "g", "o", "v", "y"}
    c = {"d", "e", "j", "k"}
    d = {"a", "b", "f", "g"}

    x = (a.intersection(b)).union(c)
    print(f"x = {x}")

    # Найдем дополнения множеств
    bn = u.difference(b)
    cn = u.difference(c)

    y = (a.difference(d)).union(cn.difference(bn))
    print(f"y = {y}")
```

Рисунок 2. Пример 1

```
D:\python\5\NCFU-Python_25-12-24-15-44>python Example1.py
x = {'k', 'j', 'd', 'e', 'o'}
y = {'g', 'h', 'v', 'o', 'f', 'y', 'c'}
```

Рисунок 3. Результат выполнения кода примера 1 с разными входными данными

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import sys
5  from datetime import date
6
7
8  if __name__ == "__main__":
9      # Список работников
10     workers = []
11
12     # Организовать бесконечный цикл запроса команд
13     while True:
14         # Запросить команду из терминала
15         command = input(">>> ").lower()
16
17         # Выполнить действие в соответствии с командой
18         if command == 'exit':
19             break
20
21         elif command == 'add':
22             # Запросить данные о работнике
23             name = input("Фамилия и инициалы? ")
24             post = input("Должность? ")
25             year = int(input("Год поступления? "))
26
27             # Создать словарь
28             worker = {
29                 'name': name,
30                 'post': post,
31                 'year': year,
32             }
33
34             # Добавить словарь в список
35             workers.append(worker)
36
37             # Отсортировать список в случае необходимости
38             if len(workers) > 1:
39                 workers.sort(key=lambda item: item.get('name', ''))
40
41         elif command == 'list':
42             # Заголовок таблицы
43             line = '+--{}-+--{}-+--{}-+--{}-+'.format(
44                 '-' * 4,
45                 '-' * 30,
46                 '-' * 20,
47                 '-' * 8
48             )

```

Рисунок 4. Пример 2

```

D:\python\5\NCFU-Python_25-12-24-15-44>python Example2.py
>>> help
Список команд:

add - добавить работника;
list - вывести список работников;
select <стаж> - запросить работников со стажем;
help - отобразить справку;
exit - завершить работу с программой.
>>> list
+-----+-----+-----+-----+
| № | Ф.И.О. | Должность | Год |
+-----+-----+-----+-----+
>>> add
Фамилия и инициалы? Скоробогатов В.А
Должность? Студент
Год поступления? 2024
>>> list
+-----+-----+-----+-----+
| № | Ф.И.О. | Должность | Год |
+-----+-----+-----+-----+
| 1 | Скоробогатов В.А | Студент | 2024 |
+-----+-----+-----+-----+
>>> select 2024
Работники с заданным стажем не найдены.
>>> select 1
1: Скоробогатов В.А
>>> select 2
Работники с заданным стажем не найдены.
>>> exit

```

Рисунок 5. Результат выполнения кода примера 2 с разными входными данными

```

raindance@DESKTOP-33LN9V8 MINGW64 /d/python/4/NCFU-Python_03-12-24-13-14 (master
)
$ git add .

raindance@DESKTOP-33LN9V8 MINGW64 /d/python/4/NCFU-Python_03-12-24-13-14 (master
)
$ git commit -m "Examples done"
[master c7efb41] Examples done
3 files changed, 75 insertions(+)
create mode 100644 Example1.py
create mode 100644 Example2.py
create mode 100644 Example3.py

```

Рисунок 6. Фиксация изменений в репозитории

Задание 1: подсчитать количество гласных в строке, введенной с клавиатуры и использованием множеств

```
Task1.py X
C: > Users > Я > Desktop > Универ > Питон > 5 > NCFU-Python_25-12-24-15-44 > Task1.py
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  if __name__ == "__main__":
5      input_string = input("Введите строку: ")
6      vowels = set("аеёиоуыэюяАЕЁИОУЫЭЮЯ")
7      count = sum(1 for char in input_string if char in vowels)
8      print(f"Количество гласных: {count}")
```

Рисунок 7. Код для задания 1

```
C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python Task1.py
Введите строку: Привет, меня зовут Виктор!
Количество гласных: 8

C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python Task1.py
Введите строку: У Лукоморья дуб зелёный, золотая цепь на дубе том
Количество гласных: 17
```

Рисунок 8. Результаты выполнения кода для задания 1

Задание 2: Определите общие символы в двух строках, введённых с клавиатуры.

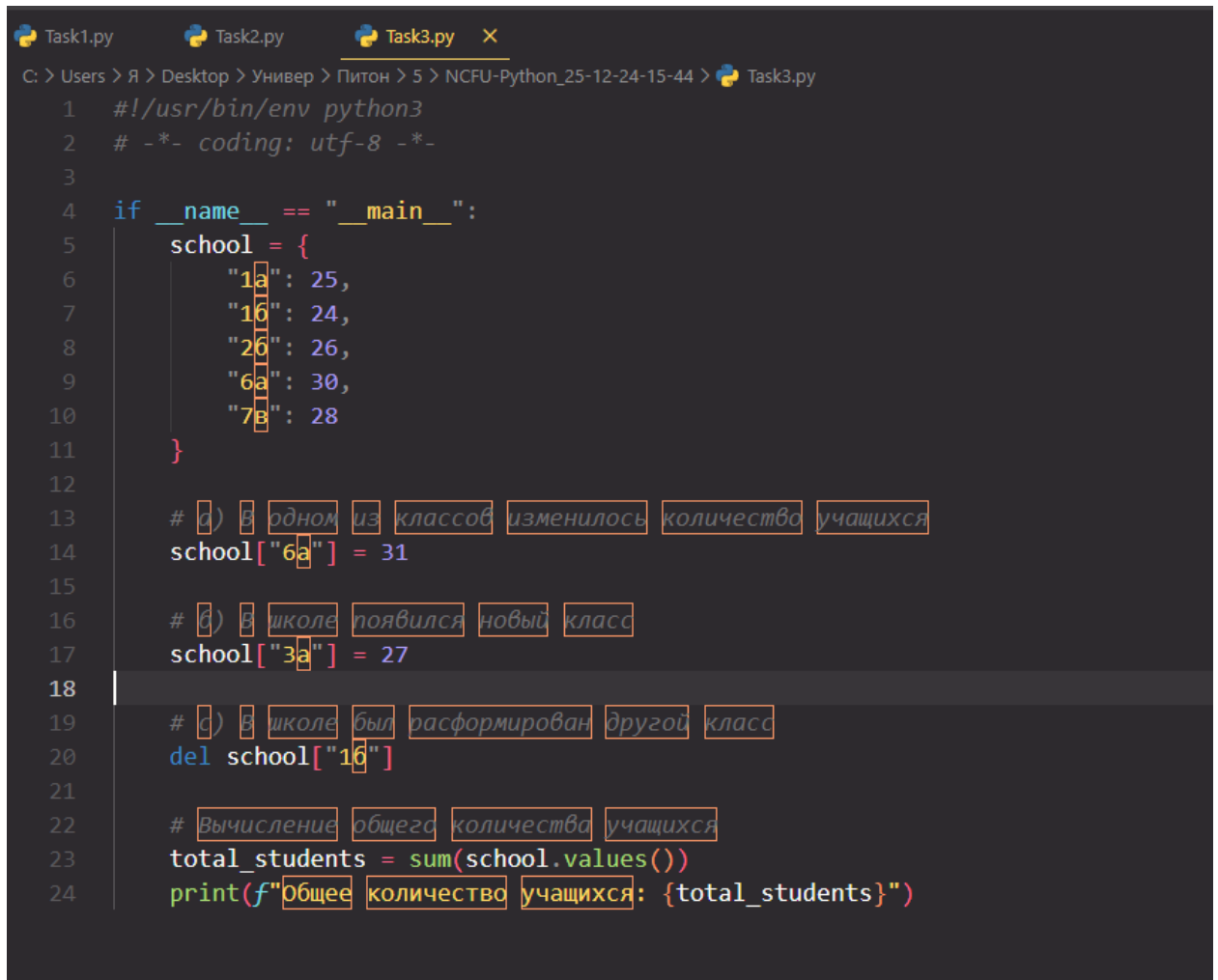
```
Task1.py Task2.py X
C: > Users > Я > Desktop > Универ > Питон > 5 > NCFU-Python_25-12-24-15-44 > Task2.py
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  if __name__ == "__main__":
5      str1 = input("Введите первую строку: ")
6      str2 = input("Введите вторую строку: ")
7      common_chars = set(str1) & set(str2)
8      print(f"Общие символы: {common_chars}")
```

Рисунок 9. Код для задания 2

```
C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python Task2.py
Введите первую строку: Привет, меня зовут Никита
Введите вторую строку: До свидания!
Общие символы: {' ', 'н', 'и', 'в', 'о', 'а', 'я'}
```

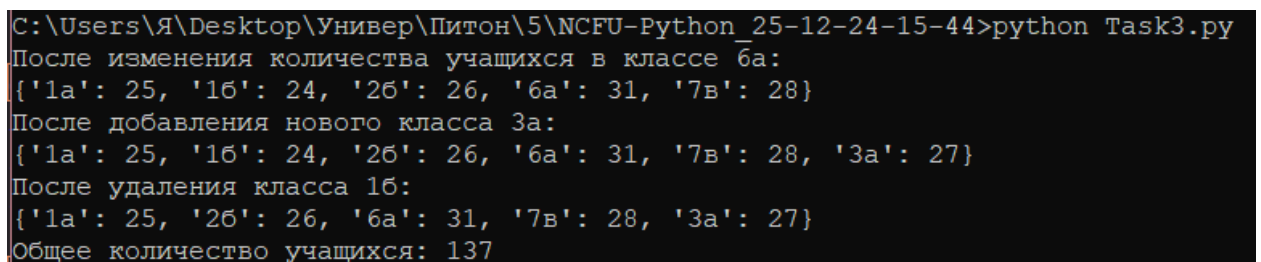
Рисунок 10. Результат выполнения кода для задания 2

Задание 3: создайте словарь, связав его с переменной `school`, и наполните данными, которые бы отражали количество учащихся в разных классах (1а, 1б, 2б, 6а, 7в и т. п.). Внесите изменения в словарь согласно следующему: а) в одном из классов изменилось количество учащихся, б) в школе появился новый класс, с) в школе был расформирован (удален) другой класс. Вычислите общее количество учащихся в школе.



```
Task1.py Task2.py Task3.py X
C: > Users > Я > Desktop > Универ > Питон > 5 > NCFU-Python_25-12-24-15-44 > Task3.py
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  if __name__ == "__main__":
5      school = {
6          "1а": 25,
7          "1б": 24,
8          "2б": 26,
9          "6а": 30,
10         "7в": 28
11     }
12
13     # а) В одном из классов изменилось количество учащихся
14     school["6а"] = 31
15
16     # б) В школе появился новый класс
17     school["3а"] = 27
18
19     # в) В школе был расформирован другой класс
20     del school["1б"]
21
22     # Вычисление общего количества учащихся
23     total_students = sum(school.values())
24     print(f"Общее количество учащихся: {total_students}")
```

Рисунок 11. Код для задания 3

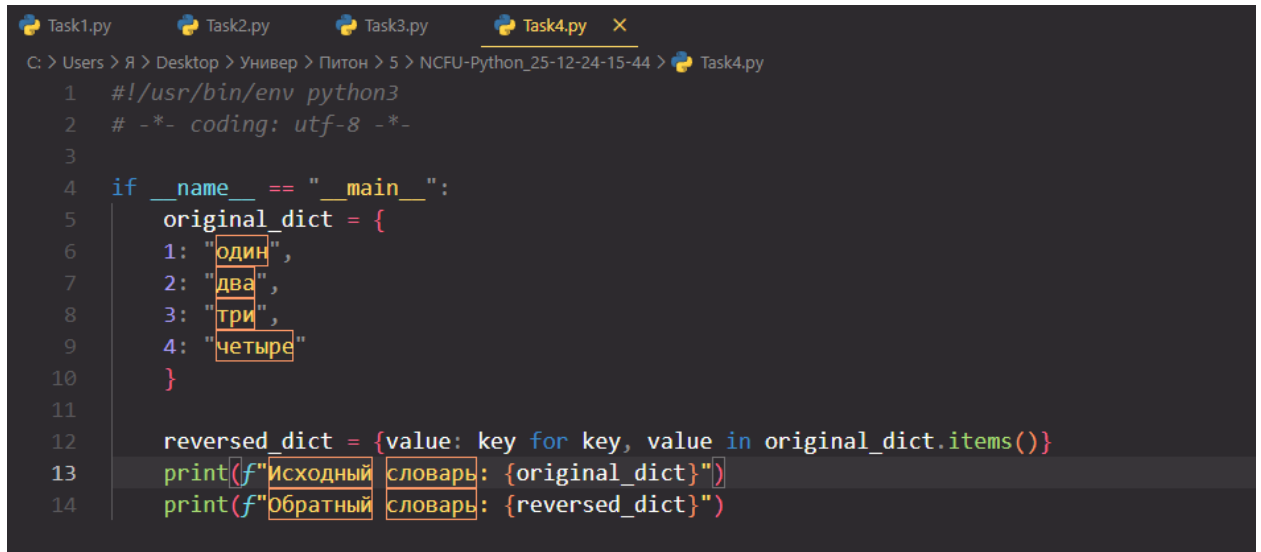


```
C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python Task3.py
После изменения количества учащихся в классе 6а:
{'1а': 25, '1б': 24, '2б': 26, '6а': 31, '7в': 28}
После добавления нового класса 3а:
{'1а': 25, '1б': 24, '2б': 26, '6а': 31, '7в': 28, '3а': 27}
После удаления класса 1б:
{'1а': 25, '2б': 26, '6а': 31, '7в': 28, '3а': 27}
Общее количество учащихся: 137
```

Рисунок 12. Результаты выполнения кода для задания 3

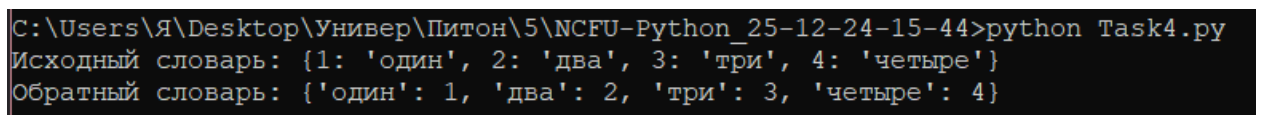
Задание 4: создайте словарь, где ключами являются числа, а значениями — строки. Примените к нему метод `items()`, с помощью

полученного объекта `dict_items` создайте новый словарь, "обратный" исходному, т. е. ключами являются строки, а значениями — числа.

A screenshot of a code editor showing a Python script named Task4.py. The script defines an original dictionary with keys 1-4 and values 'один', 'два', 'три', 'четыре'. It then creates a reversed dictionary using a dictionary comprehension and prints both. The code is as follows:

```
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  if __name__ == "__main__":
5      original_dict = {
6          1: "один",
7          2: "два",
8          3: "три",
9          4: "четыре"
10     }
11
12     reversed_dict = {value: key for key, value in original_dict.items()}
13     print(f"Исходный словарь: {original_dict}")
14     print(f"Обратный словарь: {reversed_dict}")
```

Рисунок 13. Код для задания 4

A screenshot of a terminal window showing the execution of the Python script Task4.py. The output displays the original dictionary and the reversed dictionary. The terminal text is:

```
C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python Task4.py
Исходный словарь: {1: 'один', 2: 'два', 3: 'три', 4: 'четыре'}
Обратный словарь: {'один': 1, 'два': 2, 'три': 3, 'четыре': 4}
```

Рисунок 14. Результат выполнения кода для задания 4

Индивидуальное задание 1: Определить результат выполнения операций над множествами. Считать элементы множества строками. Проверить результаты в ручную.

$$\begin{aligned}
 A &= \{c, g, h, k, y\} \\
 B &= \{a, b, k, n, u\} \\
 C &= \{i, j, o, y, z\} \\
 D &= \{a, b, f, g, y, z\} \\
 U &= \{a, b, c, g, h, k, y, z, n, u, i, j, o, z, f\} \\
 \begin{cases}
 X = (A \cup B) \cap D = \{a, b, g, y\} \\
 A \cup B = \{a, g, h, k, y, a, b, n, u\} \\
 (A \cup B) \cap D = \{a, b, g, y\}
 \end{cases} \\
 Y = (\bar{A} \cap D) \cup (\bar{C} \setminus \bar{B}) = \{a, b, f, z, k, n, u\} \\
 \bar{A} = \{a, b, n, u, i, j, o, z, f\} \\
 \bar{C} = \{a, b, c, g, h, k, n, u, f\} \\
 \bar{B} = \{c, g, h, y, i, j, o, z, f\} \\
 \bar{A} \cap D = \{a, b, f, z\} \\
 \bar{C} \setminus \bar{B} = \{a, b, g, k, n, u\} \\
 (\bar{A} \cap D) \cup (\bar{C} \setminus \bar{B}) = \{a, b, f, z, k, n, u\}
 \end{aligned}$$

Рисунок 15. Письменная проверка

```

Task1.py Task2.py Task3.py Task4.py IndTask1.py X
C: > Users > Я > Desktop > Универ > Питон > 5 > NCFU-Python_25-12-24-15-44 > IndTask1.py
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  if __name__ == "__main__":
5      # Определим универсальное множество
6      u = set("abckghkynuijzof")
7      |
8      A = {"c", "g", "h", "k", "y"}
9      B = {"a", "b", "k", "n", "u"}
10     C = {"i", "j", "o", "y", "z"}
11     D = {"a", "b", "f", "g", "y", "z"}
12
13     # X = (A ∪ B) ∩ D
14     X = (A.union(B)).intersection(D)
15     print(f"X = {X}")
16
17     # Найдем дополнения множеств
18     A_not = u.difference(A)
19     C_not = u.difference(C)
20
21     # Y = (Ā ∩ D) ∪ (C̄ \ B̄)
22     Y = (A_not.intersection(D)).union(C_not.difference(B))
23     print(f"Y = {Y}")

```

Рисунок 16. Код для индивидуального задания 1

```
C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python IndTask1.py
X = {'b', 'g', 'y', 'a'}
Y = {'c', 'g', 'h', 'a', 'z', 'f', 'b'}
```

Рисунок 17. Результат выполнения кода для индивидуального задания 1

Индивидуальное задание 2: использовать словарь, содержащий следующие ключи: фамилия и инициалы; номер группы; успеваемость (список из пяти элементов). Написать программу, выполняющую следующие действия: ввод с клавиатуры данных в список, состоящий из словарей заданной структуры; записи должны быть упорядочены по возрастанию номера группы; вывод на дисплей фамилий и номеров групп для всех студентов, включенных в массив, если средний балл студента больше 4.0; если таких студентов нет, вывести соответствующее сообщение.

```
C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44> IndTask1.py
1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import sys
5  from datetime import date
6
7
8  if __name__ == "__main__":
9      # список студентов
10     students = []
11
12     # Организовать бесконечный цикл запроса команд
13     while True:
14         # Запросить команду из терминала
15         command = input(">>> ").lower()
16
17         # Выполнить действие в соответствии с командой
18         if command == 'exit':
19             break
20
21         elif command == 'add':
22             # Запросить данные о работнике
23             name = input("Фамилия и инициалы? ")
24             group_number = input("Номер группы? ")
25             marks = [int(mark) for mark in input("Успеваемость? (Список из 5 цифр, записывать через пробел)? ").split()]
26
27             # Создать словарь
28             student = {
29                 'name': name,
30                 'group': group_number,
31                 'marks': marks,
32             }
33
34             # Добавить словарь в список
35             students.append(student)
36
37             # Отсортировать список в случае необходимости
38             if len(students) > 1:
```

Рисунок 18. Код для индивидуального задания 2

```

C:\Users\Я\Desktop\Универ\Питон\5\NCFU-Python_25-12-24-15-44>python IndTask1.py
>>> add
Фамилия и инициалы? Скоробогатов В.А.
Номер группы? 1
Успеваемость? (Список из 5 цифр, записывать через пробел)? 5 3 4 4 3
>>> add
Фамилия и инициалы? Оразов Т.И.
Номер группы? 3
Успеваемость? (Список из 5 цифр, записывать через пробел)? 4 4 4 4 4
>>> add
Фамилия и инициалы? Ромащенко Д.П.
Номер группы? 1
Успеваемость? (Список из 5 цифр, записывать через пробел)? 5 5 5 5 5
>>> list
+-----+-----+-----+-----+
| № | Ф.И.О. | Номер группы | Успеваемость |
+-----+-----+-----+-----+
| 1 | Скоробогатов В.А. | 1 | 5 3 4 4 3 |
| 2 | Ромащенко Д.П. | 1 | 5 5 5 5 5 |
| 3 | Оразов Т.И. | 3 | 4 4 4 4 4 |
+-----+-----+-----+-----+
>>> select
1: Ромащенко Д.П.
>>> help
Список команд:
add - добавить студента;
list - вывести список студентов;
select - вывод списка студентов со средним баллом больше 4.0;
help - отобразить справку;
exit - завершить работу с программой.
>>> exit

```

Рисунок 19. Результат работы кода для индивидуального задания 2

Контрольные вопросы

1) Множества – неупорядоченная совокупность уникальных значений. В качестве элемента этого набора данных могут выступать любые неизменяемые объекты, такие как числа, символы и строки. В отличие от массивов и списков, порядок следования значений не учитывается при обработке его содержимого. Над одним, а также над несколькими множествами можно выполнять ряд операций, благодаря функциям стандартной библиотеки языка программирования Python.

2) Для создания множества можно просто присвоить переменной последовательность значений, выделив их фигурными скобками. Другой вариант создания множества подразумевает использование вызова set.

3) Для проверки присутствия/отсутствия данного значения в множестве нужно использовать in.

4) Перебор всех элементов множества можно выполнить с помощью следующего кода:

```
for a in {0, 1, 2, 3}  
    print(a)
```

5) Для создания множества можно воспользоваться генератором (Set comprehensions), позволяющим заполнять списки, а также другие наборы данных с учетом неких условий.

6) Чтобы внести новые значения в множество, потребуется вызвать метод `add`

7) Для удаления элемента из множества используются следующие функции:

`remove` – удаление элемента с генерацией исключения в случае, если такого элемента нет;

`discard` – удаление элемента без генерации исключения в случае, если такого элемента нет;

`pop` – удаление первого элемента, генерируется исключение при попытке удаления из пустого множества.

Для полной очистки используется метод `clear` не принимающий аргументов

8) Чтобы объединить все элементы двух разных множеств, стоит воспользоваться методом `union` на одном из объектов

Чтобы найти пересечение двух разных множеств, следует применить функцию `intersection`, принимающую в качестве аргумента один из наборов данных

Чтобы вычислить разность для двух разных множеств, необходимо воспользоваться методом `difference`. Функция позволяет найти элементы, уникальные для второго набора данных, которых в нем нет

9) Чтобы выяснить, является ли множество `a` подмножеством `b`, нужно воспользоваться методом `issubset`.

Чтобы узнать, является ли множество *a* надмножеством *b*, необходимо вызвать метод `issuperset`

10) Множество, содержимое которого не поддается изменению имеет тип `frozenset`. Значение из этого набора нельзя удалить, как и добавить новые

11) Для преобразования множества в строку используется конкатенация текстовых значений, которую обеспечивает функция `join`. Её аргументов является набор данных в виде нескольких строк.

Чтобы получить из множества словарь, следует передать функции `dict` набор из нескольких пар значений, в каждом из которых будет находиться ключ.

Для получения списка используется вызов `list`, получающий в качестве аргумента множество *a*.

12) Словарь представляет собой структуру данных (которая еще называется ассоциативный массив), предназначенную для хранения произвольных объектов с доступом по ключу. Данные в словаре хранятся в формате ключ – значение. Словарь – это изменяемый неупорядоченный набор элементов

13) Да, функция `len()` может использоваться со словарями. Она возвращает количество пар ключ-значение в словаре.

14) Методы обхода словарей: цикл по ключам, цикл по значениям, цикл по парам ключ-значение, через метод `keys()`.

15) Способы получения значения из словаря по ключу: через квадратные скобки, метод `get()`, с проверкой наличия ключа.

16) Способы установки значения в словаре по ключу: прямое присваивание, метод `setdefault()`, метод `update()`.

17) Словарь включений — это краткий способ создания словаря с использованием цикла и условий в одной строке.

18) `Zip()` объединяет элементы нескольких итерируемых объектов в кортежи.

```
keys = ['a', 'b', 'c']
```

```
values = [1, 2, 3]
```

```
my_dict = dict(zip(keys, values))
```

```
# Результат: {'a': 1, 'b': 2, 'c': 3}
```

19) Модуль `datetime` предоставляет классы для работы с датами и временем:

- `datetime.date` — работа с датами (год, месяц, день).
- `datetime.time` — работа с временем (час, минута, секунда).
- `datetime.datetime` — комбинация даты и времени.
- `datetime.timedelta` — разница между датами/временем.
- `datetime.timezone` — работа с часовыми поясами.

Вывод: в ходе работы были приобретены навыки по работе с множествами и словарями при написании программ с помощью языка программирования Python версии 3.x.